

An Empirical Evaluation of LLM-Assisted Sketch-Based Requirements Elicitation and Prototyping

Hamdan Alabsi

halabsi@bowiestate.edu

Bowie State University

Sriram Srinivasan

Bowie State University

Rand Obeidat

Bowie State University

Nega Lakew

Bowie State University

Mangle Andrew

Bowie State University

Azene Zenebe

Bowie State University

Research Article

Keywords: Requirements Engineering (RE), Large Language Models (LLMs), Sketch-Based Elicitation, Early-Stage Prototyping

Posted Date: January 30th, 2026

DOI: <https://doi.org/10.21203/rs.3.rs-8475358/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

An Empirical Evaluation of LLM-Assisted Sketch-Based Requirements Elicitation and Prototyping

Abstract

Requirements elicitation is often described as the most challenging and error-prone phase of Software Engineering. Misinterpretations between stakeholders and system analysts frequently result in incomplete, ambiguous, or inconsistent requirements, which can propagate into project delays, rework, and system failure. In this paper, we investigate how Large Language Models (LLMs) can function as intelligent co-pilots in requirements engineering by transforming hand-drawn sketches into both (1) natural-language functional requirements and (2) HTML/CSS-based software prototypes. We ground this work in Communication Theory and Cognitive Fit Theory, positioning LLMs as mediating agents that enhance shared understanding and reduce representational gaps between stakeholders and developers. Through experimental prototyping and comparative evaluation, we assess requirement coverage, semantic correctness, prototype alignment, and perceived stakeholder alignment. Our results indicate that LLM-assisted elicitation improves efficiency, reduces ambiguity, and enables earlier validation by providing stakeholders with concrete, interactive prototypes. We conclude by discussing practical implications and outlining strategies for integrating LLMs into requirements engineering toolchains, particularly during early elicitation activities such as stakeholder interviews and on-site requirements gathering.

Keywords: Requirements Engineering (RE), Large Language Models (LLMs), Sketch-Based Elicitation, Early-Stage Prototyping

1 Introduction

Requirements Engineering (RE) lies at the heart of successful software development; however, it continues to be reported as one of the leading contributors to project failure in both industry surveys and academic studies [1–3]. Inadequate RE has been linked to costly rework, requirements drift, and stakeholder dissatisfaction, with empirical studies estimating that as much as 40–50% of project failures stem from poorly managed or misunderstood requirements [4–6]. Beyond economic impacts, researchers have emphasized that numerous accident investigations in safety-critical systems trace their

root causes back to deficiencies in requirements engineering processes, underscoring the broader societal and operational consequences of RE failures [7]. Analysts often struggle to bridge the gap between stakeholders’ informal and ambiguous ideas and the precise specifications required for implementation. Conversely, stakeholders may not recognize inconsistencies, omissions, or misaligned expectations until late in the development cycle, when corrective changes are both costly and disruptive [8].

A central challenge in requirements engineering therefore lies in transforming stakeholders’ informal and evolving ideas into clear, structured, and testable specifications that developers can implement. Analysts frequently encounter difficulty interpreting changing stakeholder visions, while stakeholders themselves may lack the technical perspective needed to validate requirements early. As a result, inconsistencies and missing functionality often surface only after substantial development effort has already been invested [8].

Visual artifacts such as sketches, wireframes, and mockups are widely used to support shared understanding during early requirements discussions. Research in human–computer interaction demonstrates that visual representations reduce cognitive load and foster richer, more productive dialogue between analysts and stakeholders during early design stages [9]. Despite these benefits, translating informal sketches into precise requirement statements or executable prototypes remains a nontrivial task. This transformation typically demands significant domain expertise, technical skill, and time, creating opportunities for miscommunication, incomplete requirements, and scope creep [10, 11].

At the same time, advances in Generative AI, particularly Large Language Models (LLMs) such as GPT-4, GPT-5, Claude, and LLaMA, demonstrate an unprecedented ability to interpret unstructured input, reason about contextual information, and generate structured outputs. These outputs include natural-language requirements, user stories, pseudo-code, and front-end software prototypes [12, 13]. Early studies in LLM-assisted software engineering suggest that such models can accelerate ideation, automate portions of requirements documentation, and provide interactive support for stakeholders who may be unfamiliar with technical notations or formal modeling languages [14, 15].

Building on this line of work, our research explores whether LLMs can function as mediators in the requirements elicitation process, as illustrated in Fig 1. Specifically, we examine whether LLMs can:

- synthesize natural-language requirements from sketches created by analysts during stakeholder interviews, thereby reducing ambiguity during early elicitation; and
- generate functional HTML/CSS prototypes that enable rapid stakeholder feedback and iterative refinement.

Accordingly, we pose the following research question:

RQ: How effective are LLMs in converting sketches into natural-language requirements and interactive prototypes to support requirements elicitation?

This research question investigates whether transforming sketches into structured requirements and subsequently translating them into executable prototypes can serve as an effective approach to requirements elicitation. Such an approach reduces the

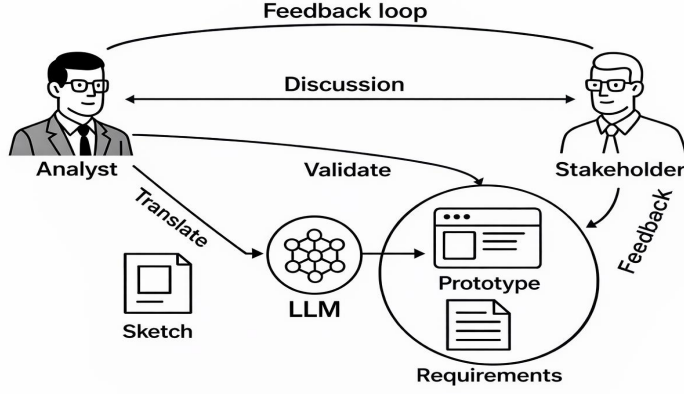


Fig. 1 LLM-assisted requirements elicitation and validation process. Analysts translate stakeholder sketches into structured artifacts using a large language model, enabling prototype and requirement generation, continuous discussion, and iterative feedback.

communication gap between analysts and stakeholders and supports early validation, thereby contributing to improved cost efficiency and shorter development timelines. To examine this premise, the paper empirically evaluates the effectiveness of LLM-supported sketch-to-prototype workflows within the requirements engineering process.

In response to this research question, this paper makes four primary contributions to requirements engineering research and practice. First, it introduces an LLM-assisted, sketch-based requirements elicitation and prototyping pipeline that transforms low-fidelity, informal sketches into both natural-language requirements and executable HTML/CSS prototypes. Second, it presents a theoretically grounded, multi-metric evaluation framework, operationalized through the Elicited Requirements Evaluation Index (EEI) and the Prototype Assessment Index (PAI), for the systematic assessment of semantic correctness, requirement coverage, traceability, and visual and functional fidelity of early-stage artifacts. Third, through a controlled empirical study with expert evaluators, the paper provides quantitative evidence of the relative strengths and limitations of contemporary LLMs, revealing a clear performance advantage of more advanced models in semantic grounding and artifact alignment. Finally, the study offers practical and methodological insights into the use of LLMs as augmentative tools in early requirements engineering, emphasizing the continued importance of human-in-the-loop validation and responsible integration into existing engineering workflows.

2 Literature Review

The requirements phase of software development consistently introduces a high risk of errors, driven by unclear stakeholder needs, shifting expectations, and organizational change. These factors contribute to requirements volatility, which remains one of

the most persistent challenges in requirements engineering. Dasanayake et al. identify ambiguous requirements, evolving stakeholder demands, and dynamic contexts as key drivers of this volatility. As a result, analysts are often forced to pursue moving targets and clarify vague or incomplete information, making the elicitation process highly error-prone [16]. Miscommunication during early requirements gathering further exacerbates these challenges, frequently leading development teams to implement incorrect features, incur costly rework, and experience scope creep and project delays [17]. Even when teams employ established RE techniques (such as structured interviews, facilitated workshops, and UML-based modeling) these processes remain labor-intensive and often produce incomplete or inconsistent specifications [4]. Existing studies indicate that the absence of robust communication practices and proactive requirements management allows early-stage errors to cascade across the development lifecycle and negatively affect project outcomes.

Against this backdrop, large language models (LLMs) have emerged as promising tools for supporting RE activities. Recent surveys suggest that LLMs are increasingly applied across multiple RE tasks, including requirements elicitation, specification drafting, and validation [18]. Zadehnoori et al. conducted a systematic review of LLM4RE research and reported a growing body of work using LLMs to generate natural-language requirements, analyze regulatory documents, and check specifications for consistency [19]. These developments indicate that LLMs have the potential to automate or assist a wide range of RE activities, from early documentation to artifact verification. However, industrial adoption remains cautious. Norheim et al. report practitioner concerns regarding the reliability of LLM outputs, particularly the risk of hallucinations (plausible but incorrect content) when models are applied to domain-specific inputs [20]. Furthermore, aligning LLM outputs with organizational terminology and specialized domains remains challenging [21]. Despite these limitations, empirical evidence increasingly suggests that LLMs are most effective when integrated into human-in-the-loop workflows, where they function as assistants rather than autonomous decision-makers.

One of the most actively studied applications of LLMs in RE is user story generation for agile development. Lucassen et al. established foundational quality criteria for user stories, which subsequent studies have used to evaluate LLM-generated outputs [22, 23]. Quattrocchi et al. assessed GPT-generated user stories using the Quality User Story (QUS) framework and found that LLMs can produce well-structured and readable stories comparable to those authored by humans [24]. Nevertheless, inconsistencies in domain coverage and contextual accuracy persist, particularly when specialized vocabulary or implicit knowledge is required [25]. Complementing this work, Barke et al. examined developer interactions with LLM-based assistants and concluded that such tools are most effective as collaborative partners (supporting ideation, accelerating documentation, and enabling rapid iteration) rather than as replacements for human judgment [14]. This perspective aligns with broader observations that AI-assisted tools enhance human creativity and productivity without displacing human expertise [26].

Beyond elicitation and story generation, researchers have explored the use of LLMs for requirements analysis and verification. Preda et al. investigated automated

requirements coverage analysis and demonstrated that LLMs achieve high accuracy and strong recall in identifying missing low-level requirements, particularly in safety-critical contexts [27]. Similarly, Luitel et al. addressed the challenge of detecting incompleteness in natural-language requirements by leveraging BERT as an external knowledge source, showing improvements over simpler baselines [28]. Other studies emphasize the importance of prompt engineering in maximizing LLM performance. Chen et al. demonstrated that techniques such as chain-of-thought prompting and self-consistency substantially improve reasoning accuracy and robustness, especially for multimodal tasks [29]. In the RE context, Binkhonain et al. further showed that prompt-based LLMs can effectively perform requirements classification without large labeled datasets, achieving performance comparable to fine-tuned transformer models [30].

Parallel to advances in text-based requirements processing, researchers have explored methods for translating visual sketches into executable prototypes. Microsoft’s Sketch2Code system demonstrated the feasibility of converting hand-drawn UI sketches into HTML markup, enabling rapid generation of rudimentary web interfaces from visual inputs [31]. While effective for layout translation, this approach focused primarily on visual structure and did not capture underlying requirements or user intent. More recent work by Kolthoff et al. introduced GUIDE, a system combining LLMs with design tools to generate UI prototypes from high-level descriptions [32]. Although such approaches improve flexibility, critics argue that many design-to-code pipelines still neglect the semantic rationale behind UI elements. Kumar et al. note that treating visual artifacts in isolation often results in interfaces that lack traceability to the requirements they are intended to satisfy [33]. Consequently, even minor requirement changes may necessitate complete UI regeneration.

Recent research has begun to address this traceability gap. North et al. proposed a gradient-based explainability approach to trace how input requirements influence LLM-generated code, enabling identification of unmet requirements and supporting iterative refinement [34]. Despite these advances, existing sketch-to-prototype approaches rarely integrate visual designs with explicit textual requirements in a unified workflow. This disconnect limits holistic validation and consensus-building between analysts and stakeholders.

To address these limitations, researchers increasingly advocate for multi-metric evaluation frameworks when assessing LLM-assisted RE systems. Textual quality is commonly evaluated using lexical overlap metrics such as BLEU [35]; however, such measures alone cannot capture semantic fidelity. Consequently, semantic similarity metrics such as BERTScore are frequently employed [36]. For visual artifacts, layout fidelity and design consistency may be assessed using structural similarity measures, while domain-specific metrics, such as traceability indices, evaluate alignment between generated artifacts and original requirements. The use of these complementary metrics supports a holistic evaluation of linguistic accuracy, semantic alignment, visual fidelity, and requirements coverage, which is critical for assessing LLM-assisted requirements engineering systems.

3 LLMs as Translation Agents: A Theoretical View

In this study, we conceptualize large language models (LLMs) as translation agents that bridge multiple representational forms within the requirements elicitation process. Specifically, LLMs are viewed as mediators between sketches, textual requirements, and functional prototypes, strengthening shared understanding between stakeholders and analysts, as illustrated in Fig. 2. This perspective aligns with prior findings identifying communication gaps (arising from scale, temporal distance, and lack of shared understanding) as primary causes of requirements failures in large-scale software development [4]. Such gaps often lead to unmet expectations, quality issues, and wasted development effort, underscoring the importance of continuous and effective requirements communication.

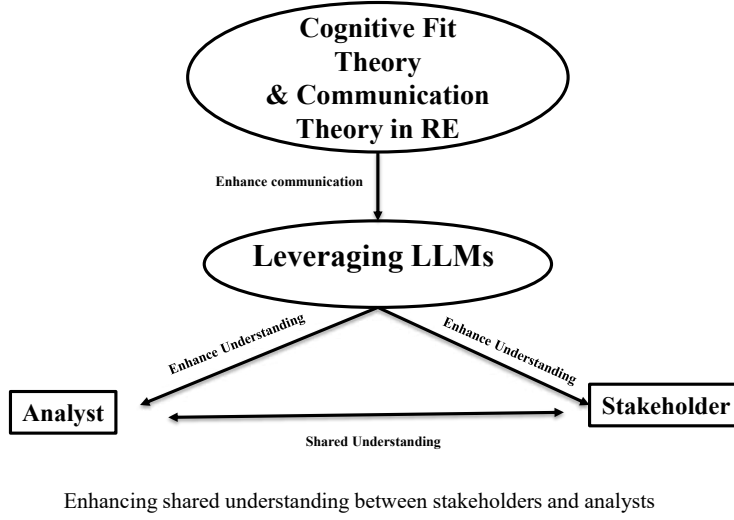


Fig. 2 Theoretical foundation of the study grounded in Cognitive Fit Theory and Communication Theory in Requirements Engineering, illustrating how shared representations support mutual understanding between analysts and stakeholders.

Our theoretical framing draws on two complementary lenses: communication theory in requirements engineering and Cognitive Fit Theory. Nuseibeh and Easterbrook describe RE as a fundamentally human-centered communication process, in which success depends on achieving shared understanding among diverse stakeholders [8]. Building on this view, we treat prototypes as boundary objects, artifacts that facilitate coordination across professional perspectives by making abstract requirements tangible [37]. Presenting requirements simultaneously as textual descriptions and visual mockups enables stakeholders, analysts, and developers to engage with a shared

representation of the envisioned system. Even low-fidelity prototypes can stimulate discussion, reveal misunderstandings, and align expectations early in the lifecycle.

We further ground our approach in Cognitive Fit Theory, which posits that problem-solving performance improves when the representation of information aligns with the cognitive demands of the task [38]. Empirical studies demonstrate that individuals reason more effectively when information is presented in formats that match their preferred modes of understanding, such as diagrams for spatial reasoning or text for procedural analysis [38, 39]. Applied to RE, this suggests that different stakeholders benefit from different representational forms.

Building on these insights, we position LLMs as mediators capable of transforming a single sketch into both textual requirements and visual prototypes in parallel. By supporting multiple representations simultaneously, LLMs accommodate diverse cognitive preferences, enhance mutual understanding, and facilitate earlier and more informed decision-making during requirements discussions. In doing so, LLMs accelerate the iterative cycle of elicitation, validation, and refinement. Ultimately, this translation-based view frames LLMs not merely as automation tools, but as communication enablers that improve the fit between how requirements are expressed, understood, and implemented.

4 Methodology

This study adopts a controlled proof-of-concept methodology to examine the feasibility and effectiveness of large language model (LLM)-assisted, sketch-based requirements elicitation and prototype generation. The methodological design draws on established practices in requirements engineering, human-computer interaction, and empirical software engineering, while remaining intentionally scoped to isolate the technical capabilities of LLMs during early-stage system design.

The methodology centers on a simulated elicitation scenario, expert-based evaluation, and theoretically grounded assessment instruments. This controlled setup enables systematic evaluation of how LLMs interpret informal visual artifacts and translate them into structured requirements and executable prototypes. The following subsections describe the scenario, materials and tools, and experimental procedure in detail.

4.1 Scenario Description

The evaluation employed a simulated yet realistic requirements elicitation scenario focused on the design of a web-based Service Portal. In this scenario, a hypothetical stakeholder requested a system capable of supporting:

1. user account creation and authentication;
2. access to available service offerings;
3. a home dashboard displaying usage statistics alongside an infographic summarizing user activity; and
4. basic navigation and contact information sections.

To capture the stakeholder’s initial conceptualization, the analyst produced a hand-drawn interface sketch representing the intended layout and functionality of the portal. The sketch included a header and logo region, a navigation bar, a statistics card, an infographic section, and side-by-side login and registration forms, as shown in Fig 3. This sketch served as the sole design artifact used to convey stakeholder intent to the LLMs, mirroring early elicitation practices in which analysts rely on lightweight visual artifacts rather than formal specifications.

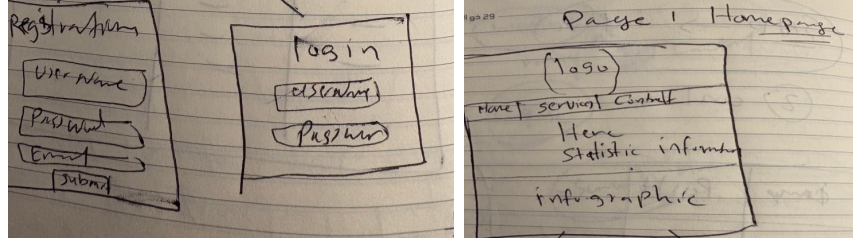


Fig. 3 Low-fidelity hand-drawn sketch of the Service Portal homepage, illustrating the registration and login interface created during the simulated requirements elicitation session. The sketch intentionally includes handwriting inconsistencies and visual noise to reflect realistic early-stage elicitation conditions.

Importantly, the sketch was intentionally created under low-fidelity conditions. It contained poor handwriting, spelling errors, uneven alignment, and visual noise, including faint text visible from the reverse side of the paper. These imperfections were deliberately introduced to reflect the informal, time-constrained nature of early requirements discussions, where sketches are often produced rapidly and lack visual polish. This design choice allowed the study to assess the robustness of LLMs when confronted with realistic, noisy inputs rather than idealized design artifacts.

4.2 Materials, Tools, and Evaluation Instruments

4.2.1 Input and Output Artifacts

The primary input artifact for this study was a hand-drawn sketch produced by the analyst during the simulated requirements elicitation session. The sketch constituted the sole representation of stakeholder intent provided to the large language models, which ensured experimental control and consistency across all conditions. The deliberate use of low-fidelity sketches is consistent with prior requirements engineering research, which emphasizes the effectiveness of informal and early-stage visual artifacts in facilitating communication and shared understanding during elicitation activities [8, 9].

The generation tools comprised three general-purpose large language models (GPT-4, GPT-5, and Google GenAI Gemini) selected for their multimodal reasoning capabilities and demonstrated proficiency in natural-language understanding and code generation [40, 41]. Each model was tasked with producing two complementary output artifacts:

1. a set of functional requirements articulated in natural language; and
2. a single-file HTML/CSS prototype representing the inferred user interface layout and functionality.

This dual-output design follows prior work advocating the parallel evaluation of requirements specifications and executable artifacts as a means of enabling early validation and reducing communication gaps between analysts and stakeholders [37, 42].

4.2.2 Evaluation Instruments and Theoretical Grounding

The evaluation framework employed in this study is grounded in established requirements quality models, international software quality standards, and usability evaluation theory. Specifically, the selected evaluation dimensions reflect classical notions of requirement correctness, completeness, clarity, and consistency, as articulated in IEEE and ISO/IEC standards [43, 44]. In addition, prototype evaluation draws on formative usability assessment practices commonly applied to early-stage interface designs [45, 46].

4.2.3 Evaluation of Elicited Requirements

The elicited requirements were assessed using a set of dimensions derived from foundational requirements engineering principles and prior empirical studies [42, 43]. Table 1 summarizes the evaluation metrics.

Table 1 Elicited Requirements Evaluation Metrics

Metric	Operational Definition
Semantic Accuracy	Degree to which generated requirements correctly reflect the intended functionality implied by the sketch, without misinterpretation or ambiguity, consistent with classical requirements correctness criteria [43, 47].
Requirement Coverage	Extent to which all functional elements and interactions depicted in the sketch are explicitly represented in the generated requirements, reflecting completeness in early-stage elicitation [42].
Novelty Precision	Degree to which generated requirements remain grounded in the sketch by avoiding unsupported, inferred, or hallucinated functionality, aligning with recent work on hallucination control in LLMs [48, 49].
Clarity and Readability	Ease with which stakeholders and analysts can understand, interpret, and validate the generated requirements during early elicitation activities [23].
Consistency	Absence of internal contradictions or logical conflicts among generated requirements statements, consistent with software quality standards [44].

The individual metric scores were aggregated into a composite Elicited Requirements Evaluation Index (EEI), calculated as the arithmetic mean of the five evaluation dimensions: Semantic Accuracy (SA), Requirement Coverage (RC), Novelty Precision (NP), Clarity and Readability (CR), and Consistency (CO). Similar composite indices have been used in prior empirical requirements engineering studies to capture the multidimensional nature of requirements quality [23, 50].

4.2.4 Evaluation of Generated Prototypes

The quality of the generated prototypes was assessed using evaluation dimensions informed by software quality models and usability engineering frameworks, particularly those articulated in ISO/IEC standards and human-centered design principles [44, 46]. Table 2 summarizes the prototype assessment metrics.

Table 2 Prototype Assessment Metrics

Metric	Operational Definition
Visual Fidelity	Degree to which the generated prototype visually reflects the layout, structure, and spatial organization depicted in the original sketch, consistent with sketch-to-interface translation approaches [31].
Functional Alignment	Extent to which the behavior and interactive elements of the prototype correspond to the intended system functionality implied by the sketch and elicited requirements [51].
Structural Code Quality	Quality of the generated HTML/CSS in terms of organization, modularity, readability, and maintainability, aligned with established software quality standards [44].
Human-Perceived Usability	Expert judgment of the prototype’s intuitiveness, ease of navigation, and overall usability during early-stage interaction, consistent with usability engineering principles [45, 46].

These dimensions were aggregated into a Prototype Assessment Index (PAI), computed as the arithmetic mean of the Visual Fidelity (VF), Functional Alignment (FA), Structural Code Quality (SCQ), and Human-Perceived Usability (HPU) scores.

4.2.5 Scoring Scheme

All evaluation metrics were rated using a five-point Likert scale (1 = very poor, 5 = excellent), consistent with established practices in expert-based software artifact evaluation [52, 53]. The standardized scoring scheme enabled consistent comparison across models and supported aggregation into the EEI and PAI indices.

4.2.6 Integrated Evaluation Rationale

In summary, the proposed requirements engineering evaluation framework captures multiple quality dimensions that are critical during early-stage elicitation. These dimensions include completeness (requirement coverage), correctness and relevance

(novelty precision and internal consistency), traceability (alignment across sketches, textual requirements, and generated prototypes), and fidelity (both visual and functional alignment). Prior work has typically examined these dimensions in isolation, for example, focusing on automated recovery of traceability links between artifacts [25] or on fine-grained visual comparison of generated user interfaces [54]. To the best of our knowledge, this study represents the first effort to integrate these complementary evaluation dimensions into a unified framework specifically designed for LLM-driven, sketch-based requirements elicitation and prototyping. The proposed approach integrates multiple quality dimensions to provide a holistic evaluation of LLM performance that captures semantic grounding, cross-artifact alignment, and usability-oriented fidelity beyond surface-level correctness.

4.3 Procedure

The experimental procedure was designed to approximate a realistic requirements elicitation workflow supported by a large language model (LLM) and to maintain sufficient experimental control for systematic evaluation. The procedure comprised six sequential stages: stakeholder stimulation (request for a meeting or elicitation session), sketch creation, sketch digitization, LLM input preparation (including sketch upload and prompt formulation), LLM-driven artifact generation, and expert evaluation and feedback, as illustrated in Fig. 4.

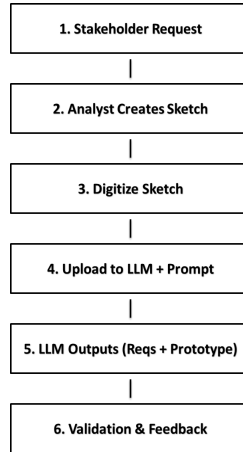


Fig. 4 LLM-assisted sketch-based requirements elicitation workflow illustrating the detailed procedural steps followed during the experimental execution. The process includes stakeholder request elicitation, analyst-driven sketch creation and digitization, submission of the sketch and prompt to the LLM, automated generation of requirements and a prototype, and subsequent validation and feedback.

First, a stakeholder role was simulated to articulate a functional request for a web-based Service Portal. Based on this interaction, the analyst translated the stakeholder’s verbal description into a hand-drawn interface sketch. The sketch served as

a concrete visual representation of stakeholder intent and functioned as the primary communication artifact for subsequent interaction with the LLMs.

The sketch was intentionally produced under realistic elicitation conditions. It was created rapidly and informally, exhibiting poor handwriting, spelling errors, uneven alignment, and minimal visual refinement. These characteristics reflect real-world analyst–stakeholder meetings, where sketches are often generated under time constraints and without design polish. In addition, the paper contained background writing and visual noise, including faint text visible beneath the sketch, which was deliberately preserved. Retaining these imperfections enabled an assessment of LLM robustness when interpreting noisy, low-fidelity inputs rather than idealized or sanitized artifacts. Example sketches of the homepage and login/registration views are shown in Fig 3.

Next, the completed sketch was digitized by photographing it and preparing the resulting image for upload. This step mirrors common requirements engineering practice, in which informal artifacts such as whiteboard drawings or paper sketches are converted into digital form for analysis, documentation, or tool-based processing.

The digitized sketch was then submitted to three multimodal generative AI models using a carefully constructed prompt. The prompt instructed each model to:

1. generate a set of natural-language functional requirements inferred from the sketch; and
2. produce a corresponding HTML/CSS prototype reflecting the sketched layout and implied functionality.

We explicitly requested the generation of both textual and executable artifacts, enabling the models to produce outputs suitable for stakeholder validation and analyst interpretation. Fig 5 presents the requirements and prototype generated by GPT-4, Fig 6 shows the corresponding outputs generated by GPT-5, and Fig 7 illustrates the outputs generated by Google GenAI Gemini.

Finally, all generated outputs were recorded, and the HTML/CSS code was rendered in a web browser to create interactive prototypes. This enabled direct comparison between the generated requirements, the original stakeholder request, and the inferred interface behavior. Expert evaluators then assessed the quality of both the elicited requirements and the generated prototypes using the evaluation instruments described in Section 4.2. Through this procedure, we demonstrate how an LLM can function as a mediating artifact that bridges informal stakeholder input and formal design representations within the requirements engineering process. Fig 8 provides an overview of the overall experimental workflow and evaluation framework.

5 Evaluation and Analysis

This section presents a comprehensive evaluation of the proposed LLM-assisted sketch-based requirements elicitation and prototype generation pipeline. The evaluation assesses not only overall model performance, but also cross-reviewer stability, differences between task types (requirements vs. prototypes), and the implications of observed performance patterns for early-stage software engineering practice.

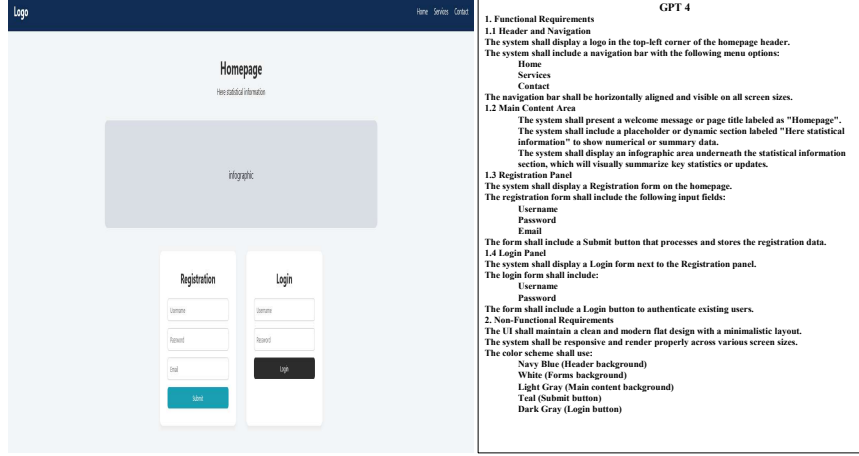


Fig. 5 LLM-generated homepage prototype and requirements synthesis from a low-fidelity hand-drawn sketch. The left panel depicts the automatically generated user interface prototype, while the right panel presents the synthesized functional and non-functional requirements expressed in structured natural language and produced by GPT-4. The figure demonstrates the model’s capability to infer interface semantics, map visual elements to requirement constructs, and transform informal sketch-based artifacts into formalized requirements and executable design representations.

Three large language models (Gemini, GPT-4, and GPT-5) were evaluated by three independent subject-matter experts using the theoretically grounded instruments introduced in Section 4.2. Each model produced two artifacts from the same low-fidelity sketch input:

1. a set of elicited functional requirements expressed in natural language, and
2. a corresponding HTML/CSS prototype.

These artifacts were evaluated using the Elicited Requirements Evaluation Index (EEI) and the Prototype Assessment Index (PAI). All scores were assigned on a five-point Likert scale (1 = very poor, 5 = excellent).

5.1 Elicited Requirements Evaluation (EEI)

Across all three evaluators, GPT-5 consistently achieved the highest EEI scores, followed by GPT-4 and Gemini. GPT-5 scores ranged from 4.7 to 4.8, GPT-4 ranged from 4.2 to 4.4, and Gemini ranged from 3.8 to 4.6. When averaged across evaluators, GPT-5 achieved an EEI of approximately 4.77, compared to 4.27 for GPT-4 and 4.13 for Gemini. This reflects a consistent performance advantage of approximately 0.5 EEI points over GPT-4 and 0.6–0.9 points over Gemini. Table 3 summarizes the evaluation results of the LLM-generated outputs based on assessments by three independent evaluators. Table 4 and Fig. 9 summarize the comparative metric-level evaluation results of the LLM-generated outputs.

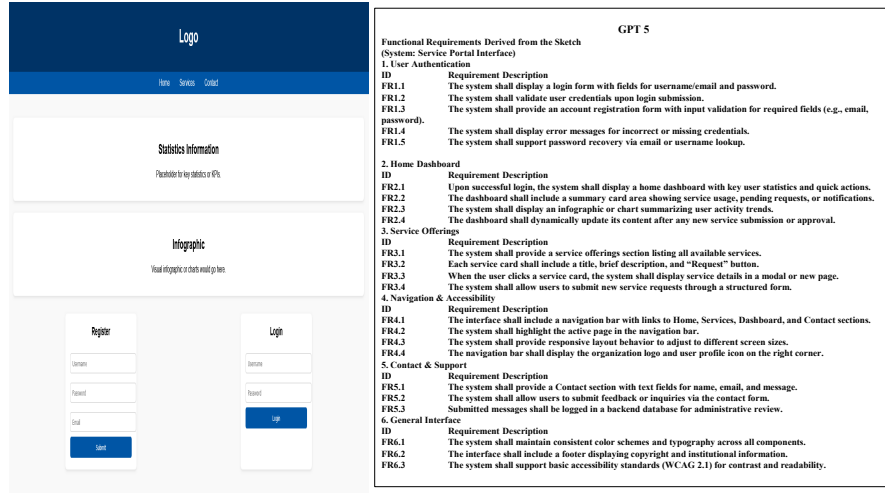


Fig. 6 LLM-generated functional requirements synthesized from a low-fidelity hand-drawn sketch. The left panel shows the inferred homepage user interface prototype, while the right panel presents a structured set of functional requirements organized by subsystem (e.g., authentication, dashboard, services, navigation, and accessibility) and expressed using standardized “shall” statements. Generated by GPT-5, the figure illustrates the model’s capability to extract interface semantics, decompose visual elements into requirement categories, and formalize informal sketch-based artifacts into traceable, system-level requirements specifications.

Table 3 Elicited Requirements Evaluation Index (EEI) Results

Model	Evaluator 1	Evaluator 2	Evaluator 3	Average EEI
Gemini	3.80	4.00	4.60	4.13
GPT-4	4.20	4.40	4.20	4.27
GPT-5	4.70	4.80	4.80	4.77

Table 4 EEI Metric Breakdown (Averaged Across Evaluators)

Model	SA	RC	NP	CR	CO
Gemini	4.06	4.33	4.00	4.17	4.33
GPT-4	3.67	4.00	4.00	4.50	5.00
GPT-5	4.17	5.00	4.67	4.67	5.00

Semantic Accuracy (SA)

Semantic accuracy exhibited the greatest variability across both models and reviewers. Gemini received scores of 3, 3, and 5 across reviewers (average ≈ 4.06), while GPT-4 received scores of 3, 4, and 4 (average ≈ 3.67). GPT-5 demonstrated more stable performance, with scores of 4, 4.5, and 4 (average ≈ 4.17). These results indicate that

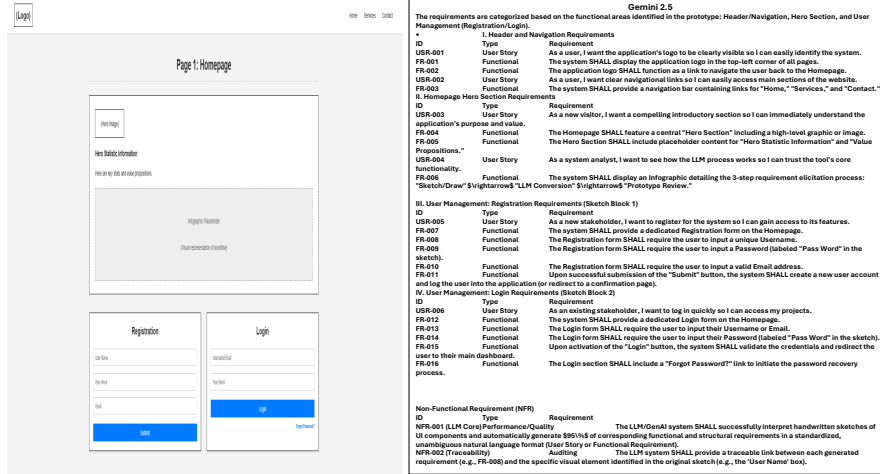


Fig. 7 LLM-generated user interface prototype and structured requirements derived from a low-fidelity hand-drawn sketch. The left panel shows the inferred homepage user interface, while the right panel presents the corresponding functional and non-functional requirements generated by Gemini 2.5. Requirements are organized by functional area (e.g., navigation, homepage sections, and user management) and expressed using standardized requirement syntax, with explicit traceability links between sketch elements and requirement identifiers.

GPT-5 improves semantic accuracy by approximately 0.5 points relative to GPT-4 and 0.1–0.8 points relative to Gemini, depending on reviewer perspective.

Requirement Coverage (RC)

Requirement coverage scores were uniformly high across models, but still revealed meaningful differentiation. GPT-5 achieved the maximum score of 5.0 from all reviewers, indicating complete coverage of sketched elements. Gemini and GPT-4 achieved average scores of approximately 4.33 and 4.0, respectively, suggesting occasional omissions of sketched functionality.

Novelty Precision (NP)

Novelty precision highlighted clear differences in hallucination control. GPT-5 received scores of 5, 4, and 5 (average ≈ 4.67), while Gemini and GPT-4 consistently scored 4.0. This represents a 0.67-point advantage for GPT-5, suggesting improved grounding and reduced introduction of unsupported requirements.

Clarity & Readability (CR)

Clarity and readability scores were high across all models, ranging from 4.0 to 5.0. GPT-5 achieved an average of approximately 4.67, compared to 4.33–4.67 for GPT-4 and 4.0–4.33 for Gemini.

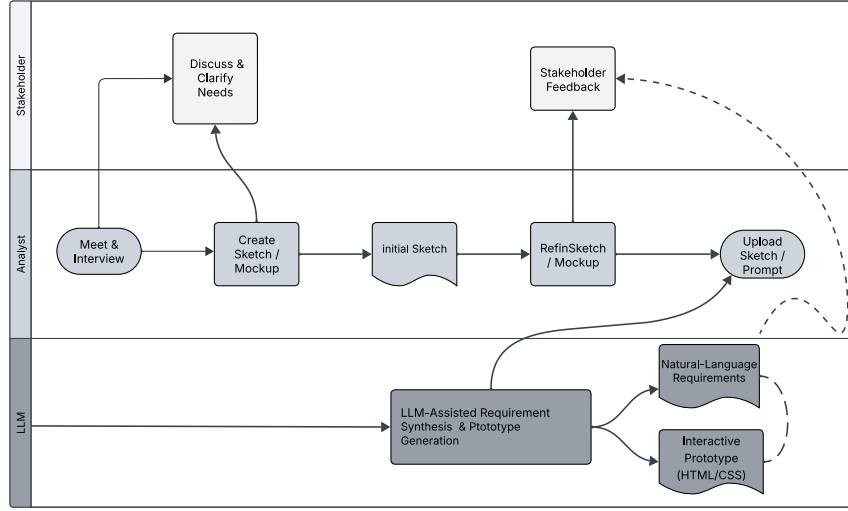


Fig. 8 Overview of the LLM-assisted sketch-based requirements elicitation process. Stakeholder requirements are captured through interviews and expressed as sketches by the analyst. These sketches are iteratively refined and provided to a large language model, which synthesizes structured natural-language requirements and produces interactive HTML/CSS prototypes. Stakeholder feedback is incorporated in a continuous loop to reduce ambiguity and mitigate costly late-stage changes.

Consistency (CO)

Consistency showed the least variation, with all models scoring 4 or higher, and GPT-4 and GPT-5 consistently receiving perfect scores.

Overall, these results indicate that while linguistic quality is strong across models, semantic accuracy, coverage, and novelty precision remain the primary differentiators in elicited requirements quality.

5.2 Prototype Evaluation (PAI)

Prototype evaluation results exhibited higher absolute scores and lower variance than requirements elicitation. GPT-5 achieved PAI scores of 5.0, 5.0, and 4.5 (average ≈ 4.83), GPT-4 achieved 4.5, 4.75, and 4.25 (average ≈ 4.50), and Gemini achieved 4.0, 4.5, and 4.25 (average ≈ 4.25). Table 5 presents the evaluation results of the LLM-generated outputs assessed by three independent evaluators. Table 6 and Fig. 10 summarize the comparative metric-level evaluation results of the LLM-generated prototypes.

Visual Fidelity (VF)

Visual fidelity scores ranged from 3 to 5 for Gemini, 4 to 4 for GPT-4, and 4 to 5 for GPT-5. Averaged across reviewers, GPT-5 achieved approximately 4.67, compared to 4.0 for GPT-4 and 3.67–4.0 for Gemini. This represents a 0.67–1.0 point improvement for GPT-5 over Gemini.

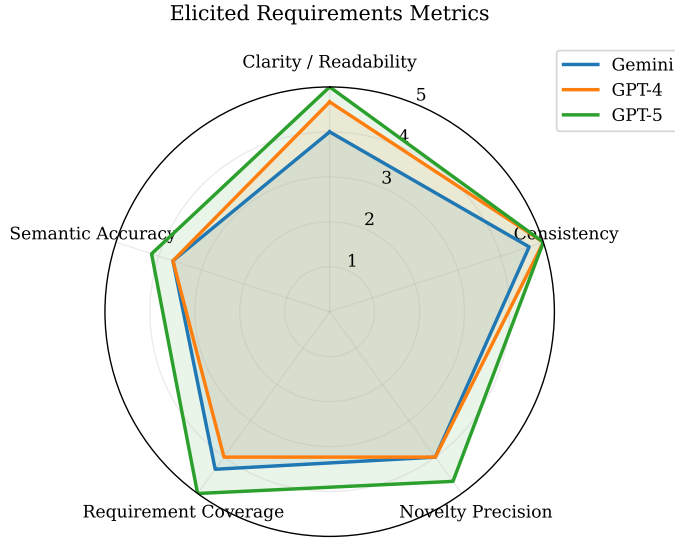


Fig. 9 Comparative evaluation of elicited requirements quality across LLMs. The radar chart reports evaluator scores for five assessment metrics (clarity/readability, consistency, novelty precision, requirement coverage, and semantic accuracy) comparing Gemini, GPT-4, and GPT-5.

Table 5 Prototype Assessment Index (PAI) Results

Model	Evaluator 1	Evaluator 2	Evaluator 3	Average PAI
Gemini	4.00	4.50	4.25	4.25
GPT-4	4.50	4.75	4.25	4.50
GPT-5	5.00	5.00	4.50	4.83

Functional Alignment (FA)

Functional alignment followed a similar pattern. GPT-5 achieved perfect scores of 5.0 across all reviewers, while GPT-4 averaged approximately 4.33 and Gemini 3.67–4.0. These results indicate that GPT-5 consistently preserved functional intent across all evaluated prototypes.

Structural Code Quality (SCQ)

Structural code quality exhibited minimal variation across models. GPT-5 and GPT-4 achieved perfect scores in two of three reviews, while Gemini received slightly lower

Table 6 PAI Metric Breakdown
(Averaged Across Evaluators)

Model	VF	FA	SCQ	HPU
Gemini	3.83	3.83	4.33	5.00
GPT-4	4.00	4.33	4.67	5.00
GPT-5	4.67	5.00	4.83	5.00

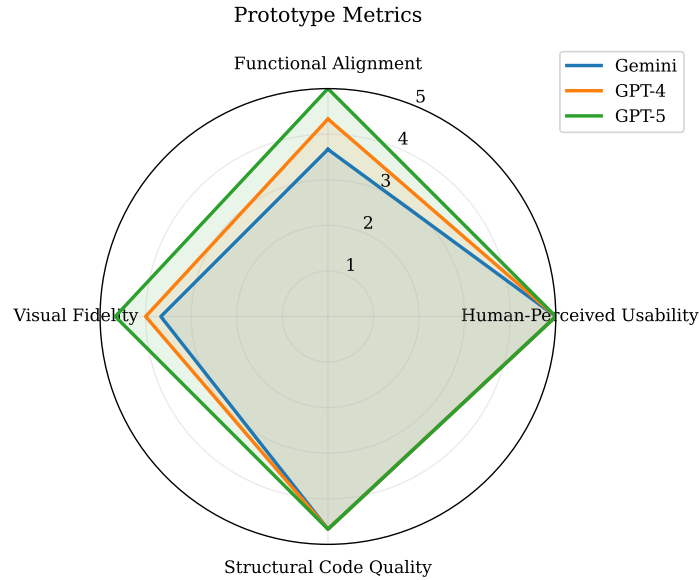


Fig. 10 Comparative evaluation of LLM-generated prototype quality. The radar chart reports evaluator scores for four prototype assessment metrics (functional alignment, human-perceived usability, structural code quality, and visual fidelity) comparing Gemini, GPT-4, and GPT-5.

scores (4–5) in one evaluation. Overall differences were within one point, indicating that all models produced structurally sound and maintainable HTML/CSS artifacts.

Human-Perceived Usability (HPU)

Human-perceived usability was uniformly rated 5.0 across all models and reviewers, indicating that all generated prototypes were considered intuitive and usable at an early design stage. This metric exhibited zero variance, highlighting the robustness of LLMs in producing user-friendly interfaces.

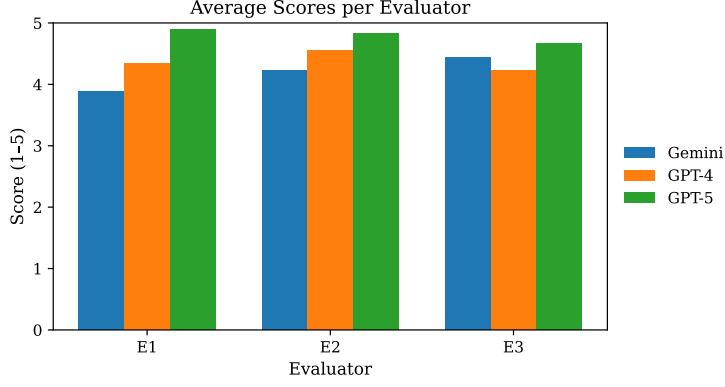


Fig. 11 Average evaluation scores per evaluator for LLM-generated outputs. The bar chart reports the mean scores assigned by three independent evaluators (E1–E3) to outputs generated by Gemini, GPT-4, and GPT-5, using a five-point Likert scale.

5.3 Cross-Reviewer Consistency and Objectivity

Despite minor differences in absolute scores, all three reviewers consistently assigned the same relative ordering to the evaluated models for both EEI and PAI, as shown in Fig. 11. Reviewer-specific variations primarily influenced semantic accuracy and structural code quality; however, the overall evaluation trends remained consistent across models.

Quantitatively, reviewer scores for the same model varied by no more than 0.8 points for EEI and 0.75 points for PAI. This high level of agreement demonstrates strong cross-reviewer consistency and supports the reliability and objectivity of the evaluation framework.

5.4 Summary of Quantitative Findings

The quantitative analysis reveals a clear and consistent performance pattern across all evaluated dimensions, as shown in Fig. 12. GPT-5 achieves higher elicited requirements quality than both GPT-4 and Gemini, with mean EEI score differences of approximately 0.4–0.6 points relative to GPT-4 and 0.6–0.9 points relative to Gemini. These performance differences persist across semantic accuracy, requirement coverage, and novelty precision, indicating systematic improvements across multiple evaluation dimensions rather than isolated metric-level gains.

Across all models, prototype generation consistently outperforms requirements elicitation, with PAI scores exceeding EEI scores by approximately 0.4–0.7 points. This pattern suggests that the structural constraints inherent in UI and code generation reduce ambiguity and stabilize model output, whereas requirements elicitation remains more sensitive to interpretation.

The largest performance gaps emerge in semantic accuracy and requirement coverage, underscoring the difficulty of translating informal, low-fidelity sketches into precise functional specifications. In contrast, prototype usability and structural correctness remain consistently high across all models and reviewers, indicating that

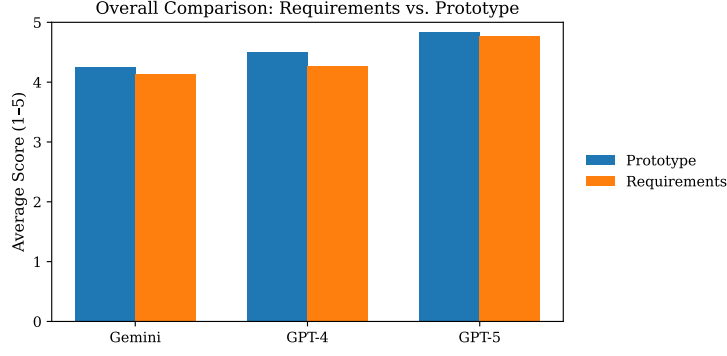


Fig. 12 Overall comparison of average evaluation scores for requirements and prototype outputs across LLMs. The bar chart reports aggregated mean scores for requirements quality and prototype quality for Gemini, GPT-4, and GPT-5, enabling a direct comparison of model performance across the two output categories.

LLMs reliably generate interpretable and executable interfaces even when semantic understanding varies.

Overall, the results demonstrate that state-of-the-art LLMs, particularly GPT-5, significantly improve the quality of early-stage requirements elicitation and prototype generation. However, they also highlight the enduring role of human judgment in validating semantic fidelity and ensuring alignment with stakeholder intent.

6 Discussion

This study set out to examine whether large language models (LLMs) can effectively support early-stage requirements elicitation and prototype generation when the primary input is an informal, low-fidelity sketch. The evaluation results yield several important insights into the capabilities and limitations of LLM-assisted workflows, as well as their implications for requirements engineering (RE) research and practice.

First, the findings demonstrate that contemporary LLMs can reliably translate informal sketches into structured design artifacts, including both natural-language requirements and executable HTML/CSS prototypes. Across evaluators and metrics, all evaluated models produced coherent outputs that captured the major functional elements implied by the sketches. This result is noteworthy because early-stage requirements elicitation has traditionally relied on analysts’ manual interpretation of ambiguous, incomplete, and often inconsistent artifacts [1, 8]. The results suggest that LLMs can act as effective intermediaries in this process, helping analysts rapidly externalize stakeholder intent into tangible and reviewable artifacts.

At the same time, the evaluation reveals a clear distinction between semantic reasoning tasks and structural generation tasks. Prototype generation consistently outperformed requirements elicitation across all models, exhibiting higher absolute scores and lower variance. This pattern aligns with longstanding insights in software engineering, which emphasize that the most difficult aspects of system development stem from conceptual and semantic complexity rather than from construction mechanics

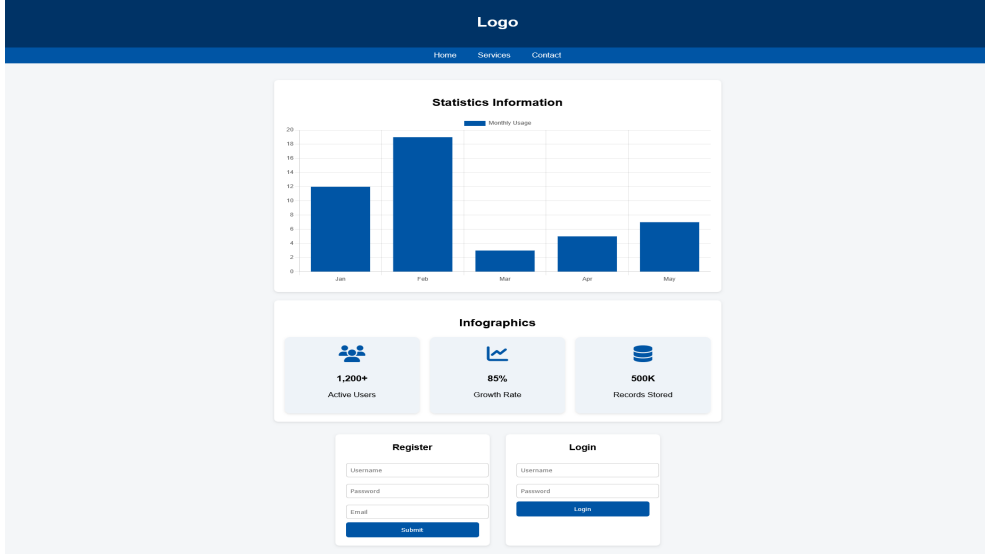


Fig. 13 LLM-generated homepage prototype derived from the low-fidelity sketch. The rendered interface includes a header with navigation, a statistics visualization panel, an infographic summary section, and user authentication components (registration and login forms), illustrating the end-to-end prototype output used during the experimental execution.

[55]. The generation of structured artifacts, such as user interfaces, benefits from established layout conventions and syntactic constraints, whereas deriving requirements from sketches involves interpreting intent, implicit assumptions, and domain context, which requires deeper semantic reasoning. From a cognitive perspective, sketches and prototypes serve as external representations that offload cognitive effort, but semantic interpretation still requires human sensemaking [56].

The consistently superior performance of GPT-5 across both elicited requirements and prototype metrics further indicates that model capability directly influences the reliability of early-stage automation. GPT-5 not only achieved higher average scores, but also demonstrated improved control over hallucinated requirements and more complete coverage of sketched elements. In addition, GPT-5 frequently generated follow-up questions after producing the requested outputs and proposed richer, more interactive interface elements (e.g., enhanced infographics), indicating a more proactive and dialog-oriented interaction style, as illustrated in Fig 13. These behaviors were observed consistently across reviewers, reinforcing the conclusion that the performance gains reflect genuine model differences rather than evaluator bias. From a practical standpoint, this finding implies that organizations considering LLM-assisted requirements workflows must carefully evaluate model selection, as not all LLMs offer equivalent levels of semantic robustness.

Another key insight concerns the continued importance of human expertise. Despite the strong performance of advanced LLMs, the evaluation underscores that human judgment remains essential, particularly during requirements elicitation. While the

models consistently produced readable and well-structured requirements, readability alone does not guarantee correctness. Reviewers identified subtle semantic mismatches and inferred functionality that, although plausible, was not explicitly supported by the sketch. This observation aligns with established human–AI interaction guidelines, which emphasize that AI systems should augment human reasoning rather than replace it, especially in tasks involving interpretation, ambiguity, and accountability [57]. In requirements engineering contexts where outputs may carry contractual, regulatory, or safety implications, retaining humans as the final decision-makers remains essential.

The uniformly high scores in human-perceived usability and structural code quality also carry important implications. Even when semantic interpretation varied, the generated prototypes were consistently rated as usable and structurally sound. This suggests that LLM-generated prototypes can function effectively as boundary objects, shared artifacts that facilitate communication across stakeholders with different expertise and perspectives [58]. Design research similarly highlights the role of early prototypes as cognitive artifacts that support collaborative reasoning and alignment [59]. By rapidly producing visual and interactive representations of stakeholder intent, LLMs can enable earlier validation and iterative refinement, potentially reducing misunderstandings before they propagate into later development stages.

These findings also resonate with scenario-based design principles, which emphasize grounding requirements in concrete usage situations rather than abstract specifications [60]. LLM-generated prototypes allow stakeholders to interact with scenarios early, making omissions and inconsistencies more visible than textual descriptions alone. However, scenario-based methods also caution that plausibility does not guarantee completeness or correctness, reinforcing the need for expert validation and iterative feedback.

From a methodological perspective, the reviewer-based evaluation and observed cross-reviewer consistency strengthen the credibility of these findings. Although expert assessment inherently involves some subjectivity, structured expert evaluation is widely accepted as an appropriate method for assessing early-stage and exploratory artifacts that are not yet suitable for large-scale user testing [52]. The consistent ranking of models and limited variance across reviewers indicate that the observed trends are stable and reproducible.

From an engineering practice standpoint, the emphasis on early validation and stakeholder communication aligns with modern RE standards. ISO/IEC/IEEE 29148 explicitly highlights the importance of early requirements validation, stakeholder involvement, and the use of representations tailored to audience needs [61]. LLM-assisted sketch-to-prototype workflows offer a practical mechanism for operationalizing these principles within contemporary development environments.

Finally, this study contributes to the broader discourse on LLMs in requirements engineering by highlighting both opportunity and caution. On one hand, LLM-assisted sketch-based workflows offer clear efficiency gains by accelerating the transformation of informal inputs into actionable artifacts. On the other hand, decades of software engineering research caution against fully automating activities rooted in semantic

interpretation and judgment [55, 62]. The results therefore support a hybrid, human-in-the-loop model in which LLMs act as powerful translation and communication aids rather than autonomous requirements engineers.

In summary, the value of LLMs in early-stage requirements engineering lies not in replacing analysts, but in augmenting their ability to reason, communicate, and iterate rapidly. When combined with human expertise, structured evaluation frameworks, and established RE practices, LLM-assisted sketch-based workflows have the potential to meaningfully enhance the speed and quality of early design activities while preserving accountability and semantic correctness.

7 Threats to Validity and Limitations

As with any empirical study involving expert judgment and rapidly evolving artificial intelligence technologies, the findings of this work are subject to several limitations and potential threats to validity. We discuss these considerations transparently to clarify the scope within which the results should be interpreted.

Internal validity may be affected by the inherently subjective nature of expert-based evaluation. Although the study employed structured rubrics and independent scoring by three subject-matter experts, assessments of semantic accuracy, requirement coverage, and usability inevitably involve human interpretation. To mitigate this risk, we provided standardized evaluation guidelines and assessed inter-rater reliability. The resulting measures indicate acceptable internal consistency (Cronbach’s $\alpha = 0.756$) and moderate agreement ($\text{ICC}(2, k) = 0.518$). While these values are consistent with prior studies evaluating early-stage design artifacts, they do not eliminate subjectivity entirely.

Construct validity concerns whether the selected metrics adequately capture the quality of elicited requirements and generated prototypes. The evaluation instruments used in this study are grounded in established requirements engineering principles and software quality models. However, they primarily emphasize early-stage design attributes such as semantic correctness, coverage, clarity, and usability. As a result, important downstream concerns (including scalability, performance, security, regulatory compliance, and long-term maintainability) are beyond the scope of this evaluation. Accordingly, the reported results reflect early design effectiveness rather than end-to-end system readiness.

External validity is limited by the proof-of-concept nature of the study. The evaluation focused on a single service portal scenario using one low-fidelity sketch. Although the sketch was intentionally noisy to approximate realistic elicitation conditions, the findings may not generalize to more complex domains, richer interaction flows, or sketches with substantially different visual styles or levels of abstraction. In addition, the study involved a small number of expert evaluators, all with strong technical backgrounds, which may not fully represent the perspectives of non-technical stakeholders or end users.

Conclusion validity is constrained by the exploratory design and limited sample size. While the analysis reveals consistent numerical trends and stable model rankings across reviewers, the study does not aim to establish statistical significance in

a hypothesis-testing sense. Instead, the results are intended to demonstrate feasibility, comparative performance, and qualitative patterns rather than definitive causal claims.

Finally, technological validity represents an inherent limitation of research involving large language models. The study evaluates specific versions of GPT-4, GPT-5, and Gemini at a particular point in time. As these models continue to evolve, their capabilities (especially in multimodal reasoning, grounding, and interaction) may change. Consequently, the relative performance differences observed in this study may narrow, widen, or shift in future model iterations.

Overall, these limitations do not undermine the core contributions of the study, but rather define its scope. The results should be interpreted as evidence that LLM-assisted, sketch-based workflows are feasible and promising for early-stage requirements engineering, while still requiring human oversight and further validation in larger, more diverse, and more realistic settings.

8 Future Work

This study demonstrates the feasibility and potential of LLM-assisted sketch-based requirements elicitation and prototype generation and identifies several promising directions for future research.

First, future work will scale the evaluation to more complex and diverse scenarios. The current proof-of-concept focuses on a single service portal with a limited set of functional requirements. Subsequent studies will examine sketches with greater structural complexity, multiple interaction flows, and domain-specific constraints (e.g., healthcare, finance, or cybersecurity). These scenarios will enable assessment of how well LLMs generalize beyond relatively simple layouts and whether semantic accuracy degrades as functional interdependencies increase.

Second, future research will expand the participant pool and interaction structure. While this study intentionally adopted a minimal stakeholder-analyst configuration to isolate technical feasibility, real-world requirements engineering often involves multiple stakeholders with competing priorities and varying levels of expertise. We plan to investigate multi-stakeholder elicitation settings in which LLMs must reconcile divergent inputs, track evolving requirements, and support negotiation and refinement across iterative cycles.

Third, we aim to incorporate iterative, human-in-the-loop feedback mechanisms. In the present study, LLMs generated requirements and prototypes in a single pass. Future work will explore multi-round interactions in which analysts and stakeholders iteratively correct, refine, and constrain model outputs. This line of investigation will help determine whether structured feedback loops can systematically reduce semantic errors, improve requirement coverage, and mitigate hallucinated functionality over successive iterations.

Fourth, future studies will examine automated traceability and consistency checking across sketches, requirements, and prototypes. While this study relies on expert judgment to assess alignment, future work will explore algorithmic approaches for detecting mismatches, omissions, and contradictions across artifacts. Such methods

could further reduce analyst workload and support continuous validation during early development phases.

Fifth, we plan to explore alternative evaluation methodologies and quantitative validation strategies. Although expert-based evaluation is appropriate for early-stage artifacts, future research will incorporate user studies, task-based usability testing, and statistical hypothesis testing with larger samples. In addition, separating reliability analysis by task type (requirements versus prototypes) and by metric category may provide more granular insight into evaluation stability and robustness.

Finally, we will investigate the integration of LLM-assisted sketch-based workflows into existing requirements engineering toolchains. Embedding these capabilities into modeling environments, requirements management systems, or collaborative white-board platforms could enable more seamless adoption in practice. Such integration will also allow examination of how LLM assistance influences analyst behavior, communication dynamics, and decision-making over longer project lifecycles.

In summary, future work will move beyond proof-of-concept validation toward scalable, interactive, and tool-supported workflows, with the goal of establishing principled, responsible, and effective use of LLMs in real-world requirements engineering contexts.

9 Conclusion

In this paper, we investigated the feasibility and effectiveness of LLM-assisted sketch-based requirements elicitation and prototype generation as a means of supporting early-stage software engineering activities. Motivated by the informal, ambiguous, and rapidly evolving nature of initial stakeholder communication, we proposed and evaluated an end-to-end pipeline in which low-fidelity sketches serve as the primary input for generating both natural-language requirements and executable HTML/CSS prototypes.

The evaluation, grounded in structured expert assessment and theoretically informed metrics, demonstrates that contemporary large language models can translate informal visual artifacts into coherent and usable design outputs. Across three independent evaluators and multiple quality dimensions, we observed consistent performance trends indicating that more advanced models, particularly GPT-5, achieve higher levels of semantic accuracy, requirement coverage, and functional alignment. The results further reveal that prototype generation is more robust and less variable than requirements elicitation, suggesting that structured artifact synthesis currently represents a more mature application domain for LLM assistance than semantic interpretation.

Importantly, the findings reinforce that LLMs should be viewed as augmentative tools rather than autonomous agents in requirements engineering. Although the models consistently produced readable, well-structured requirements and highly usable prototypes, expert review remained essential for validating semantic correctness and identifying subtle misalignments with stakeholder intent. This observation underscores the continued centrality of human expertise, particularly during the earliest phases of system design where ambiguity and implicit assumptions are unavoidable.

Beyond technical feasibility, this work contributes methodologically by introducing a theoretically grounded evaluation framework (the Elicited Requirements Evaluation Index and the Prototype Assessment Index) and demonstrating their applicability for expert-based assessment of early-stage artifacts. The inclusion of inter-rater reliability analysis further strengthens the credibility, transparency, and reproducibility of the evaluation approach.

Overall, the results suggest that LLM-assisted, sketch-based workflows offer a promising pathway for accelerating early requirements elicitation and prototyping, improving communication among stakeholders, and enabling rapid iteration on design concepts. At the same time, the study delineates clear boundaries for responsible adoption, emphasizing the need for human-in-the-loop validation and careful integration into existing engineering practices.

We conclude that large language models, when combined with structured evaluation frameworks and expert oversight, have the potential to meaningfully enhance early-stage requirements engineering. As these models continue to evolve, future research and practice should focus on scalable validation, iterative interaction, and seamless tool integration to fully realize their benefits while preserving accountability and semantic integrity.

References

- [1] Hofmann, H.F., Lehner, F.: Requirements engineering as a success factor in software projects. *IEEE Software* **18**(4), 58–66 (2001)
- [2] Hove, S.E., Anda, B.: Experiences from conducting semi-structured interviews in empirical software engineering research. In: *Proceedings of the 11th IEEE International Software Metrics Symposium (METRICS 2005)*, pp. 1–10. IEEE, ??? (2005)
- [3] Somangshu: The True Cost of Bad Requirements: How to Avoid Project Failures with Impact Analysis. <https://brewstudio.medium.com/%the-true-cost-of-bad-requirements-how-to-avoid-project-failures-%with-impact-analysis-f420cb7a292c>. Accessed: 2025-01-01 (2024)
- [4] Bjarnason, E., Runeson, P., Borg, M., Unterkalmsteiner, M., Engström, E., Regnell, B., Feldt, R.: Challenges and practices in aligning requirements with verification and validation. *Empirical Software Engineering* **19**(6), 1809–1855 (2014)
- [5] Sogeke, B.: How poor requirements lead to catastrophic software failures. *Medium* (2025). <https://medium.com/@sogekebosun/how-poor-requirements-lead-to-catastrophic-software-failures-aab25399f398>
- [6] Iqbal, J., Ahmad, R.B., Khan, M., Alyahya, S., Nasir, M.H.N., Akhunzada, A., Shoaib, M.: Requirements engineering issues causing software development outsourcing failure. *PLOS ONE* **15**(4), 0229785 (2020)

- [7] Johnson, C.W., Holloway, C.M.: Questioning the role of requirements engineering in safety-critical software failures. In: 1st IET International Conference on System Safety, pp. 352–361 (2006)
- [8] Nuseibeh, B., Easterbrook, S.: Requirements engineering: A roadmap. In: Proceedings of the Conference on the Future of Software Engineering, pp. 35–46 (2000)
- [9] Scaife, M., Rogers, Y.: External cognition: How do graphical representations work? *International Journal of Human-Computer Studies* **45**(2), 185–213 (1996)
- [10] Niknafs, A., Berry, D.M.: The impact of domain knowledge on the effectiveness of requirements idea generation during requirements elicitation. In: 2012 20th IEEE International Requirements Engineering Conference (RE), pp. 181–190 (2012). IEEE
- [11] Niknafs, A., Berry, D.M.: The impact of domain knowledge on the effectiveness of requirements engineering activities. *Empirical Software Engineering* **22**(1), 80–133 (2017)
- [12] Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Sutton, C.: Program synthesis with large language models. arXiv preprint arXiv:2108.07732 (2021)
- [13] Belzner, L., Gabor, T., Wirsing, M.: Large language model assisted software engineering. In: International Conference on Bridging the Gap Between AI and Reality, pp. 355–374. Springer, ??? (2023)
- [14] Barke, S., James, M.B., Polikarpova, N.: Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* **7**(OOPSLA1), 85–111 (2023)
- [15] Cheng, H., Husen, J.H., Lu, Y., Racharak, T., Yoshioka, N., Ubayashi, N., Washizaki, H.: Generative ai for requirements engineering: A systematic literature review. arXiv preprint arXiv:2409.06741 (2024)
- [16] Dasanayake, S., Aaramaa, S., Markkula, J., Oivo, M.: Impact of requirements volatility on software architecture (2019)
- [17] Beaton, C.: Why requirements gathering challenges can lead to overruns (2022). <https://www.requiment.com/why-requirements-gathering-challenges-can-lead-to-overruns/>
- [18] Khan, J.A., Qayyum, S., Dar, H.S.: Large language model for requirements engineering: A systematic literature review (2025)

- [19] Zadenoori, M.A., Dabrowski, J., Alhoshan, W., Zhao, L., Ferrari, A.: Large language models (llms) for requirements engineering (re): A systematic literature review. arXiv preprint arXiv:2509.11446 (2025)
- [20] Norheim, J.J., Rebentisch, E.: Structuring natural language requirements with large language models. In: IEEE 32nd International Requirements Engineering Conference Workshops (REW), pp. 68–71 (2024)
- [21] Hemmat, A., Sharbaf, M., Kolahdouz-Rahimi, S., Lano, K., Tehrani, S.Y.: Research directions for using llm in software requirement engineering. *Frontiers in Computer Science* **7**, 1519437 (2025)
- [22] Lucassen, G., Dalpiaz, F., Werf, J.M.E., Brinkkemper, S.: Forging high-quality user stories. In: IEEE International Requirements Engineering Conference, pp. 126–135 (2015)
- [23] Lucassen, G., Dalpiaz, F., Werf, J.M.E., Brinkkemper, S.: Improving agile requirements: The quality user story framework. *Requirements Engineering* **21**(3), 383–403 (2016)
- [24] Quattrocchi, G., Pasquale, L., Spoletini, P., Baresi, L.: Can llms generate user stories and assess their quality? arXiv preprint arXiv:2507.15157 (2025)
- [25] Hassine, J.: An llm-based approach to recover traceability links between security requirements and goal models. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, pp. 643–651 (2024)
- [26] Amquest Education: How AI will change software engineering in the next 5 years. <https://amquesteducation.com/how-ai-will-change-software-engineering-in-the-next-5-years/>. Industry report / online article, accessed January 2025 (2025)
- [27] Preda, A.-R., Mayr-Dorn, C., Mashkoo, A., Egyed, A.: Supporting high-level to low-level requirements coverage reviewing with large language models. In: MSR 2024 (2024). <https://doi.org/10.1145/3643991.3644922>
- [28] Luitel, D., Hassani, S., Sabetzadeh, M.: Improving requirements completeness through large language models. *Requirements Engineering* **29**, 73–95 (2024)
- [29] Chen, B., Zhang, Z., Langrené, N., Zhu, S.: Unleashing the potential of prompt engineering for large language models. *Patterns* **6**(6), 101260 (2025)
- [30] Binkhonain, M., Alfayez, R.: Are prompts all you need? evaluating prompt-based llms for software requirements classification. *Requirements Engineering* **30**, 423–443 (2025)

- [31] Microsoft Azure: Turn your whiteboard sketches to working code in seconds with Sketch2Code (2018). <https://azure.microsoft.com/en-us/blog/turn-your-whiteboard-sketches-to-working-code-in-seconds-with-sketch2code/>
- [32] Kolthoff, K., Kretzer, F., Bartelt, C., Maedche, A., Ponzetto, S.P.: Guide: Llm-driven gui generation decomposition for automated prototyping. In: ICSE Companion (2025)
- [33] Kumar, M.: Bridging the gap: Integrating ai assisted code suggestions in ui workflows. *International Journal of Engineering Technology Research & Management* **9**(3), 71–80 (2023)
- [34] North, M., Atapour-Abarghouei, A., Bencomo, N.: Code gradients: Towards automated traceability of llm-generated code. In: *IEEE International Requirements Engineering Conference*, pp. 321–329 (2024). <https://doi.org/10.1109/RE59067.2024.00038>
- [35] Papineni, K., Roukos, S., Ward, T., Zhu, W.: Bleu: A method for automatic evaluation of machine translation. *ACL*, 311–318 (2002) <https://doi.org/10.3115/1073083.1073135>
- [36] Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y.: Bertscore: Evaluating text generation with bert. In: *ICLR* (2020)
- [37] Carlile, P.R.: A pragmatic view of knowledge and boundaries. *Organization Science* **13**(4), 442–455 (2002)
- [38] Vessey, I., Galletta, D.: Cognitive fit: An empirical study of information acquisition. *Information Systems Research* **2**(1), 63–84 (1991)
- [39] Golnam, A., Tapandjieva, G., Viana, S., Wegmann, A.: Problem structuring methods in system dynamics modeling: A cognitive fit perspective. In: *International Conference of the System Dynamics Society* (2011)
- [40] Belzner, L., Gabor, T., Wirsing, M.: Generative AI in software engineering: Opportunities and challenges. *IEEE Software* (2023)
- [41] Cheng, L., et al.: Large language models for software engineering: A systematic review. *ACM Computing Surveys* (2024)
- [42] Bjarnason, E., Wnuk, K., Regnell, B.: Requirements are slipping through the gaps: A case study on causes and effects of communication gaps in large-scale software development. *Requirements Engineering* **16**(1), 37–55 (2011)
- [43] Ieee recommended practice for software requirements specifications. *IEEE Std 830-1998*, 1–40 (1998) <https://doi.org/10.1109/IEEESTD.1998.88286>
- [44] ISO/IEC: ISO/IEC 25010: Systems and software quality models. *International*

Organization for Standardization (2011)

- [45] Nielsen, J.: Usability Engineering. Morgan Kaufmann, San Francisco, CA, USA (1994)
- [46] ISO: ISO 9241-11: Ergonomics of human-system interaction—Usability. International Organization for Standardization (2018)
- [47] Wiegers, K.E., Beatty, J.: Software Requirements, 3rd edn. Microsoft Press, Redmond, WA, USA (2013)
- [48] Ji, Z., et al.: Survey of hallucination in natural language generation. ACM Computing Surveys (2023)
- [49] Hemmat, M., et al.: Hallucination risks in large language models. IEEE Software (2025)
- [50] Quattrocchi, G., *et al.*: Evaluating LLM-generated user stories. In: IEEE International Requirements Engineering Conference (RE) (2025)
- [51] Sommerville, I.: Software Engineering, 10 edn. Pearson, Boston, MA, USA (2016)
- [52] Kitchenham, B., Pfleeger, S.L., Pickard, L.: Preliminary guidelines for empirical research in software engineering. IEEE Transactions on Software Engineering **28**(8), 721–734 (2002)
- [53] Barke, S., James, M.B., Polikarpova, N.: Grounded copilot: How programmers interact with code-generating models. In: Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE) (2023)
- [54] Awal, R., Massoud, M., Feizi, A., Li, Z., Wang, S., Pal, C., Agrawal, A., Vazquez, D., Reddy, S., Rodriguez, J.A., *et al.*: Webmmu: A benchmark for multimodal multilingual website understanding and code generation. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 25129–25156 (2025)
- [55] Brooks, F.P.: No silver bullet: Essence and accidents of software engineering. IEEE Computer **20**(4), 10–19 (1987) <https://doi.org/10.1109/MC.1987.1663532>
- [56] Hutchins, E.: Cognition in the Wild. MIT Press, Cambridge, MA, USA (1995)
- [57] Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P.N., Inkpen, K., *et al.*: Guidelines for human-ai interaction. In: Proceedings of the 2019 Chi Conference on Human Factors in Computing Systems, pp. 1–13 (2019)
- [58] Star, S.L., Griesemer, J.R.: Institutional ecology, translations’ and boundary objects: Amateurs and professionals in berkeley’s museum of vertebrate zoology,

- 1907-39. *Social studies of science* **19**(3), 387–420 (1989)
- [59] Norman, D.: *The Design of Everyday Things: Revised and Expanded Edition*. Basic books, ??? (2013)
 - [60] Carroll, J.M.: *Making Use: Scenario-based Design of Human-computer Interactions*. MIT press, ??? (2003)
 - [61] Iso/iec/ieee international standard - systems and software engineering – life cycle processes – requirements engineering. ISO/IEC/IEEE 29148:2018(E), 1–104 (2018) <https://doi.org/10.1109/IEEESTD.2018.8559686>
 - [62] Parnas, D.L., Clements, P.C.: A rational design process: How and why to fake it. *IEEE transactions on software engineering* (2), 251–257 (1986)